

SBIAHU CSE 114 Fall 2025

Final Exam Practice Questions

Final exam will be on 01/17/2026, 10:20 – 12:20

- It is an in-person, paper based test, you don't need any computer/laptop for the test.
- **Please be on time; students arriving more than 10 minutes late will NOT be admitted to the exam.**
- The exam is CLOSED-BOOK, CLOSED-NOTES. No electronic or manual assistive devices (other than a writing implement) may be used during the exam, so leave laptops, tablets, smartphones, calculators, smartwatch, slide rules, etc., at home (or, at least, turn them off and stow them in your backpacks so that they are out of reach and cannot be seen or consulted during the exam).
- Final answers should be written using (ideally) non-erasable blue or black ink. DO NOT use pink, red, or purple ink under any circumstances!

Final Exam Main Focus:

- ArrayList
- Exceptions and Text I/O
- Abstract Classes and Interface

You also need the knowledge from previous chapters. More emphasis will be on the contents after Midterm 2.

First 6-7 questions of the test have sub-parts which will be short to medium size questions. These questions are TRUE/FALSE, multiple choice, fill in the blanks, rewrite the code segment, code analysis (what will be the output of this program) types.

In the last 3-4 problems (questions) you have to write the complete programs. In these questions you must properly have the import statement (if required), must define the class properly, must have correct main method header, should have proper variable declaration and initialization, proper statement termination, etc.

Do not miss any exams! Make-up exams will be given only in extenuating circumstances at the discretion of the instructor, subject to the university policies. Students who miss an exam for a valid reason must contact the instructor immediately to schedule a make-up exam at the earliest possible time.

Information on this page will be provided.

The string class:

- `charAt(n)`: returns the character at index n. indices are numbered from 0- (length - 1)
- `length()`: returns the number of characters contained in the current String
- `toLowerCase()`: returns all-lowercase version of the current String
- `toUpperCase()`: returns all-uppercase version of the current String
- `substring(i, j)`: returns a new String containing the characters from index i up to but not including, index j
- `substring(i)`: returns a new String containing the characters from index i up to the end of the String.
- `indexOf(s)`: returns the index of the first occurrence of the String s in the current String, or -1 if it cannot be found.
- `equals(s)`: returns true if its argument matches the current String exactly, including case.(compare to the `equalsIgnoreCase()` method)

The Math class:

- `abs(x)`: returns the absolute value of x.
- `max(x, y)` : returns the larger of x and y.
- `min(x, y)`: returns the minimum of x and y.
- `pow(b, e)`: raises be to the e-th power.
- `random()`: returns a random double from 0.0 up to (not including) 1.
- `round(x)`: rounds x to the nearest integer value.
- `sqrt(x)`: returns the square root of x, where $x \geq 0$.

java.util.ArrayList	
+ArrayList()	Creates an empty list.
+add(o: Object) : void	Appends a new element o at the end of this list.
+add(index: int, o: Object) : void	Adds a new element o at the specified index in this list.
+clear(): void	Removes all the elements from this list.
+contains(o: Object): boolean	Returns true if this list contains the element o.
+get(index: int) : Object	Returns the element from this list at the specified index.
+indexOf(o: Object) : int	Returns the index of the first matching element in this list.
+isEmpty(): boolean	Returns true if this list contains no elements.
+lastIndexOf(o: Object) : int	Returns the index of the last matching element in this list.
+remove(o: Object): boolean	Removes the element o from this list.
+size(): int	Returns the number of elements in this list.
+remove(index: int) : Object	Removes the element at the specified index.
+set(index: int, o: Object) : Object	Sets the element at the specified index.

1. Pick the correct option for the following questions (a-i are True/False questions):
 - a. (2 points) Java class inheritance supports multiple inheritance.
 - b. (2 points) ArrayList provides constant time access and update like an array data structure
 - c. (2 points) Constructors of the parent class are inherited in the child class
 - d. (2 points) An abstract class can have a non-abstract method
 - e. (2 points) An abstract class can be used as a reference type in polymorphism.
 - f. (2 points) ArithmeticException is an example of a checked exception.
 - g. (2 points) IOException is an example of a checked exception.
 - h. (2 points) An interface can be used as a reference type in polymorphism.
 - i. (2 points) An interface can have a non-abstract method.
 - j. (2 points) When a class overrides a method, what object reference is used to call the method inherited from the super class?
A. Inherited B. super C. class D. methodName
2. (4 points)
 - a. Explain why the following program displays [1, 3] rather than [2, 3].

```
import java.util.ArrayList;
public class Problem {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.remove(1);
        System.out.println(list);
    }
}
```
 - b. How do you remove integer 3 from the list, when the list has elements 1, 2 and 3 in this order (without using remove with index)?
3. (4 points) What is the difference between the method overloading and method overriding in Java?

4. (4 points) You coded the following in the Test.java class in order to output the smallest element in the array a:

```
public class Test {  
    public static void main(String[] args) {  
        int[][] a = {{9, 8, 7, 6},{10, 20, 30, 40}};  
        int min = a[0][0];  
        for(int i = 1; i < a.length; i++) {  
            for(int j = 0; j < a[i].length; j++) {  
                if(a[i][j] < min)  
                    min = a[i][j];  
            }  
        }  
        System.out.println("The minimum is:-->" + min);  
    }  
}
```

The program compiles properly, but when you run, you get the following output:

The minimum is 9.

You expect the value of min to be 6. Explain what the problem is and how to fix it.

5. Complete the following program:

```
public class Quiz {  
    public static void main(String[] args) {  
        B b = new B();  
        b.m1();  
    }  
}  
  
interface A{  
    public void m1();  
}  
// complete the following code, where B implements interface A  
class B .....{  
.....  
}
```

6. What will be the output of the following program:

```
import java.util.Scanner;
public class Problem {
    public static void main(String[] args) {
        int x, y;
        Scanner stdin = new Scanner(System.in);
        System.out.println("enter the value of x:->");
        x = stdin.nextInt();
        System.out.println("enter the value of y:->");
        y = stdin.nextInt();
        stdin.close();
        try {
            System.out.println(x);
            System.out.println(x/y);
            System.out.println(y);
        }
        catch(Exception ex) {
            System.out.println(" An exception!");
        }
        finally {
            System.out.println("Finally block executing!");
        }
        System.out.println("Last statement of the program!");
    }
}
```

- a. When $x = 4$ and $y = 2$
- b. When $x = 2$ and $y = 0$

7. What will be the output of the following program:

```
public class QuizException {
    public static void main(String[] args) {
        try {
            method1();
            System.out.println("statement1");
        }
        catch(Exception ex) {
            System.out.println("caught in main method");
        }
        System.out.println("statement2");
    }
    public static void method1() {
        try {
            method2();
            System.out.println("statement3");
        }
        catch(ArithmeticException ex) {
            System.out.println("Arithmetic Exception: method1()");
        }
        System.out.println("statement4");
    }
    public static void method2() {
        try {
            method3();
            System.out.println("statement5");
        }
        catch(IndexOutOfBoundsException ex) {
            System.out.println("Index out of bounds: method2()");
        }
        System.out.println("statement6");
    }
    public static void method3() {
        int[] x = new int[5];
        int z = 1/0;
    }
}
```

8. What is wrong with the following program? Fix the problem.

```
public class ClassA {
    private int x;
    public ClassA(int x) {
        this.x = x;
    }
    public static void main(String[] args) {
        ClassB b = new ClassB();
    }
}

class ClassB extends ClassA{
    public ClassB() {
    }
}
```

9. Will the following program compile (no compiler error)? If yes then why and if no then how will you fix it?

```
public class Problem {
    public static void main(String[] args){
        Integer n1 = new Integer(3);
        Object n2 = new Integer(4);
        System.out.println(n1.compareTo(n2));
    }
}
```

10. Will the following program compile (no compiler error)? If yes then why and if no then how will you fix it?

```
public class Test {
    public static void main(String[] args) {
        Persons[] persons = {new Persons(3), new Persons(4), new
Persons(1)};
        java.util.Arrays.sort(persons);
    }
}

class Persons{
    private int id;
    Persons(int id){
        this.id = id;
    }
}
```

11. (10 points) Write down a program in Java using a text file for the following tasks:
- Generate a text file named “cities.txt” that contains ten strings.
 - Read the contents of the file created in part a) and write these strings in the reverse order in another file named “output.txt”.

Hint: You may use an ArrayList of String.

12. (10 points) Implement a class **Square** that extends the abstract **GeometricObject** class provided below. Then, write a test program (in a separate tester class) that prompts the user to enter the **length** of a side of the square, a **color**, and a **boolean** value to indicate whether the square is **filled**. The program should create a **Square** object using the entered side **length**, and set the **color** and **filled** properties accordingly. Finally, display the **area**, **perimeter**, **color**, and a **true** or **false** value indicating whether the square is filled.

Declare appropriate data fields, and implement constructors along with getter and setter methods as needed.

```
public abstract class GeometricObject {
    private String color = "white";
    private boolean filled;
    private java.util.Date dateCreated;
    protected GeometricObject() {
        dateCreated = new java.util.Date();
    }
    protected GeometricObject(String color, boolean filled) {
        dateCreated = new java.util.Date();
        this.color = color;
        this.filled = filled;
    }
    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }
    public java.util.Date getDateCreated() { return dateCreated; }
    public String toString() {
        return "created on " + dateCreated + "\ncolor: " + color +
            " and filled: " + filled;
    }
    /** Abstract method getArea */
    public abstract double getArea();
    /** Abstract method getPerimeter */
    public abstract double getPerimeter();
}
```